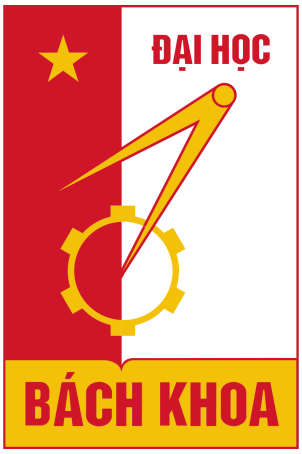
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY



PROJECT I REPORT

Topic

RUBIK'S CUBE WEB SOLVER
USING THE KOCIEMBA ALGORITHM

Course: Project I

Instructor: Mai Xuan Ngoc

Student: Le The Nhat Anh

Student ID: 202416770

Class: IT - E15 01 K69

Hanoi, July 2, 2026

TABLE OF CONTENTS

1 Introduction **1**

1.1 The Machine Perspective 1

1.2 The Core Challenge 2

2 Singmaster Move Notation **3**

3 Web Demo **4**

3.1 Scramble Input Mode 4

3.2 Manual Painting Mode 4

3.3 Solution Display and Playback 5

4 Background Theory **7**

4.1 Thistlethwaite's Algorithm (1981) 7

4.2 4-List Algorithm (Shamir's Algorithm, 1989) 7

4.3 Korf's Algorithm (1997) 7

4.4 Deep Learning Approaches (DeepCubeA) 7

4.5 Kociemba's Algorithm (1992) — Used in this project 8

5 Kociemba Algorithm **9**

5.1 Introduction to the Two-Phase Algorithm 9

5.2 Cubie Representation of the Rubik's Cube 10

5.2.1 Corner Cubies 10

5.2.2 Edge Cubies 10

5.2.3 Applying Moves on Cubie Representation 11

5.2.4 Validity Checks (Parity) 11

5.3 The Two Phases in Detail 12

5.3.1 Phase 1: Reaching Group G1 12

5.3.2 Phase 2: Solving Within G1 13

5.4 Lookup Tables . . . 13
5.4.1 Move Tables . . . 13
5.4.2 Pruning Tables . . . 14
5.5 IDA* Search Algorithm . . . 16
5.6 Suboptimal Search Strategy . . . 18

6 Web Architecture **20**
6.1 Backend Architecture . . . 20
6.1.1 Directory Structure . . . 20
6.1.2 Key Architectural Features . . . 20
6.1.3 Backend Request Flow . . . 22
6.2 Frontend Architecture . . . 22
6.2.1 Frontend File Structure . . . 22
6.2.2 Module Responsibilities . . . 23
6.2.3 Frontend Exception Handling . . . 23
6.2.4 Frontend Architecture Diagram . . . 24
6.2.5 Frontend User Flow (Color Painting Mode) . . . 25
6.3 Overall Application Flow . . . 25

7 Future Development **27**

1 Introduction

The Rubik's Cube, invented by Erno Rubik in 1974. It consists of a $3 \times 3 \times 3$ arrangement of smaller cubes (called "cubies"), with each face covered by 9 colored stickers in one of six colors. The goal of this puzzle is simple: starting from a scrambled state, return each face to a single, uniform color. Over the years, many variants have evolved from the original 3x3 cube, such as the 2x2, 4x4, 5x5, Megaminx, Skewb, and Square-1. Today, the Rubik's Cube has become one of the most iconic and best-selling puzzles around the world.

Figure 1.1 Rubik Cube

Despite its simple appearance, the Rubik's Cube poses a surprisingly deep combinatorial challenge. The original cube 3x3 has 43,252,003,274,489,856,000 (approximately 43 quintillion) possible states — only one of which is the solved state. Yet, through years of practice and the development of highly efficient human solving methods (CFOP, Roux, ZZ), elite competitive speedsolvers — called speedcubers — can solve the cube in under 6–7 seconds. The current world record stands at approximately 2.76 seconds, set by Teodor Zajder (Poland) at the GLS Big Cubes Gdansk event.

1.1 The Machine Perspective

While humans rely on pattern recognition and memorized algorithms which is favorable for fingertrick, computers approach the problem very differently. For a machine, the goal is not only to find a solution, but to find a solution with the fewest possible moves. This matters in several real-world contexts:

- Official scramble generation: World Cube Association (WCA), the international governing body that regulates and organizes official speedcubing competitions, use a method called Random-State Scrambling to generate official scrambles for competitions. Unlike random sequences of moves, the computer picks a random valid configuration of the cube, calculates the shortest path to solve it, and then reverses those moves to create an official scramble.

- Proof of God's Number: It has been mathematically proven (in 2010, by Rokicki et al.) that any configuration of the 3×3 Rubik's Cube can be solved in at most 20 moves in the Half-Turn Metric — this number is known as "God's Number." Finding and verifying this required algorithms capable of solving all 43 quintillion states efficiently.
- Robotics: Robot arms designed to solve the cube must use efficient algorithms to minimize motor operations and solve time.

1.2 The Core Challenge

The scale of the state space makes a brute-force search entirely infeasible. If a computer were to explore states at a rate of 1 billion per second, exhaustively searching all 43 quintillion configurations would take over 1,300 years. Therefore, a clever algorithmic strategy is essential — one that dramatically narrows the search space by leveraging the mathematical structure of the cube group.

This report presents a web-based Rubik's Cube solver that tackles this challenge using the Kociemba Two-Phase Algorithm, one of the most practically efficient algorithms for solving the 3×3 cube.

2 Singmaster Move Notation

To denote a sequence of moves on the $3 \times 3 \times 3$ Rubik's Cube, this project uses Singmaster notation, which was developed by David Singmaster.

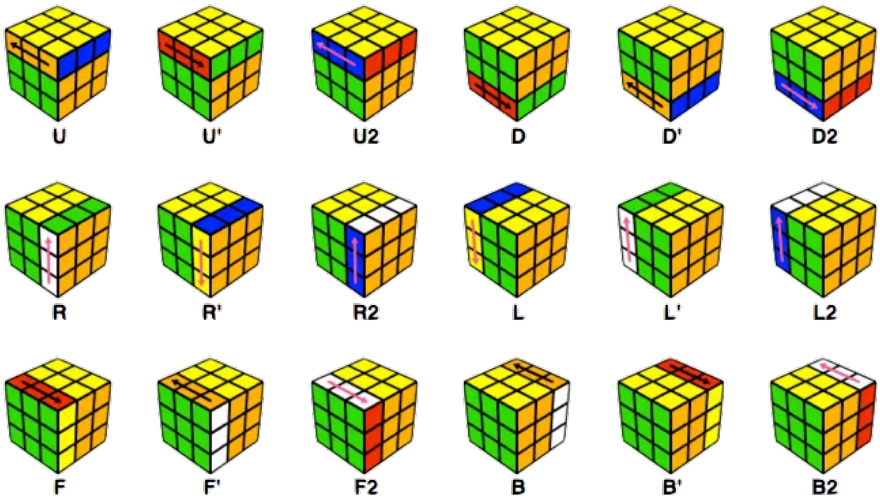


Figure 2.1 Move Notation

The letters L, R, F, B, U, and D indicate a clockwise quarter turn of the Left, Right, Front, Back, Up, and Down faces, respectively.

- A half-turn (i.e. 2 quarter turns in the same direction) is indicated by appending a `2` (e.g., `U2`).
- A counterclockwise turn is indicated by appending a prime symbol (') (e.g., `U'`).

The common color convention, which is also used in this web application, is:

- U (Up): White
- D (Down): Yellow
- F (Front): Green
- B (Back): Blue
- L (Left): Orange
- R (Right): Red

3 Web Demo

This section presents the main user-facing screens of the Rubik Solver web application. The interface is designed as a single-page application with a 3D cube viewer on the left and the input/solution controls on the right. Users can either enter a scramble sequence directly or manually paint the cube's sticker colors before requesting a solution.

- Domain: https://rubiksolver.duckdns.org

3.1 Scramble Input Mode

In Scramble Mode, the user enters a standard Singmaster move sequence into the text box. The Random button can generate a random scramble automatically, while Scramble 3D applies the entered sequence to the 3D cube preview. After the cube is scrambled, the user clicks Solve Now to send the state to the backend solver.

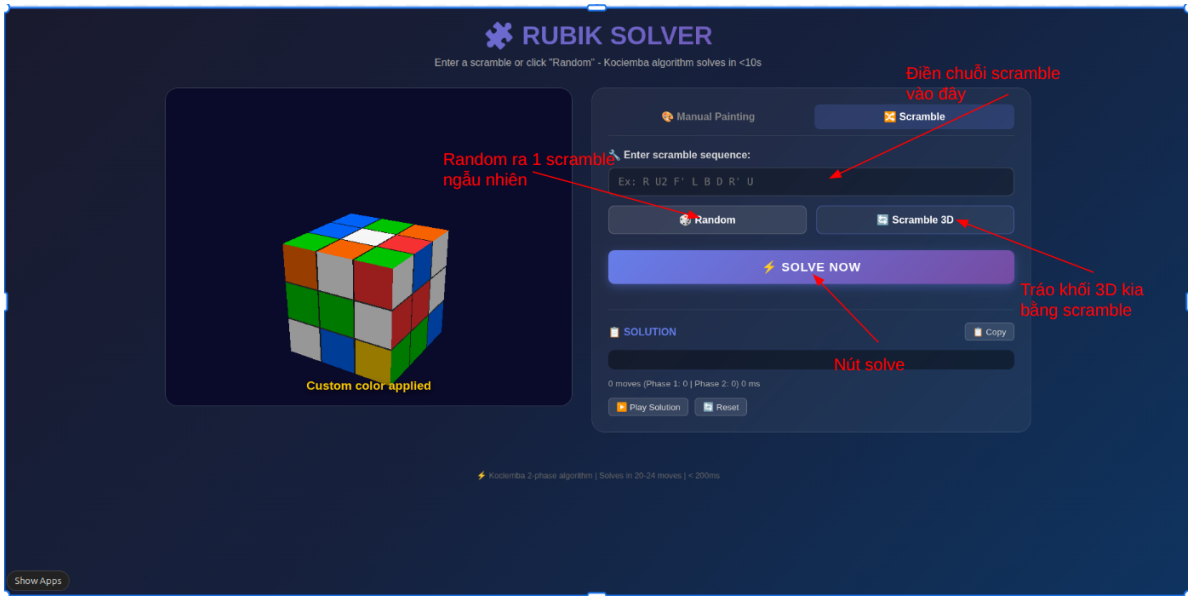


Figure 3.1 Scramble input mode demo

3.2 Manual Painting Mode

Manual Painting Mode allows the user to describe a cube state by coloring the 2D net. The user first selects one of the six cube colors, then clicks individual stickers on the map to paint them. After the 2D layout is complete, Apply to 3D updates the WebGL cube preview so the user can visually confirm the configuration before solving.

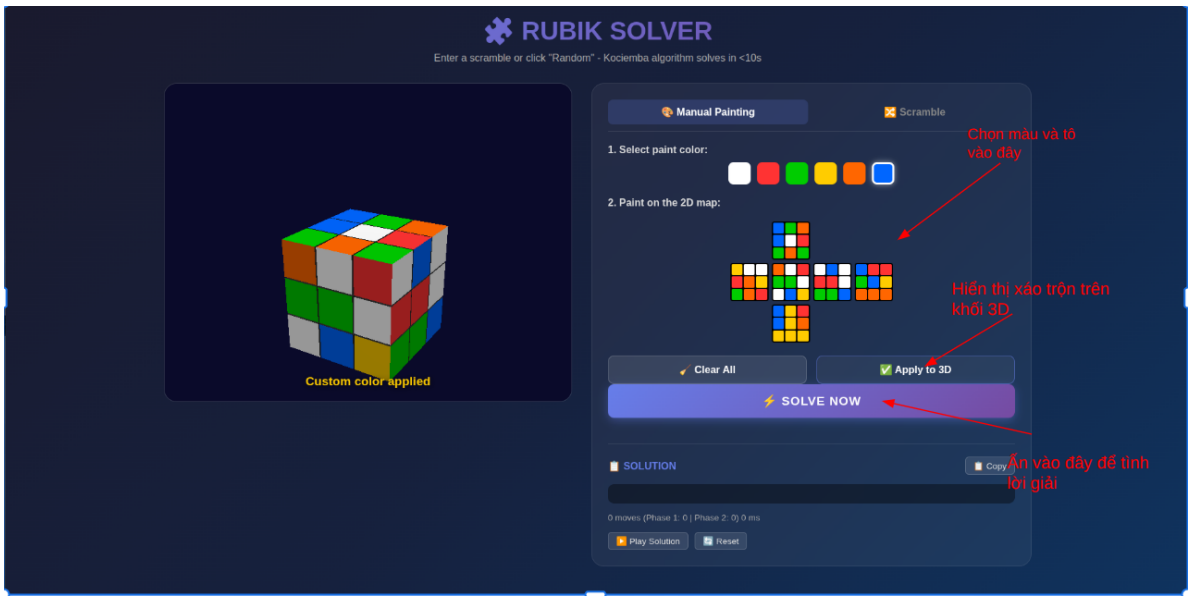


Figure 3.2 Manual painting mode demo

3.3 Solution Display and Playback

After the backend returns a solution, the interface displays the move sequence as individual move tokens, together with the total move count, Phase 1 length, Phase 2 length, and solving time. The Play Solution button animates the returned moves on the 3D cube, allowing the user to follow the solving process step by step. The Copy button makes it easy to reuse the generated solution elsewhere.

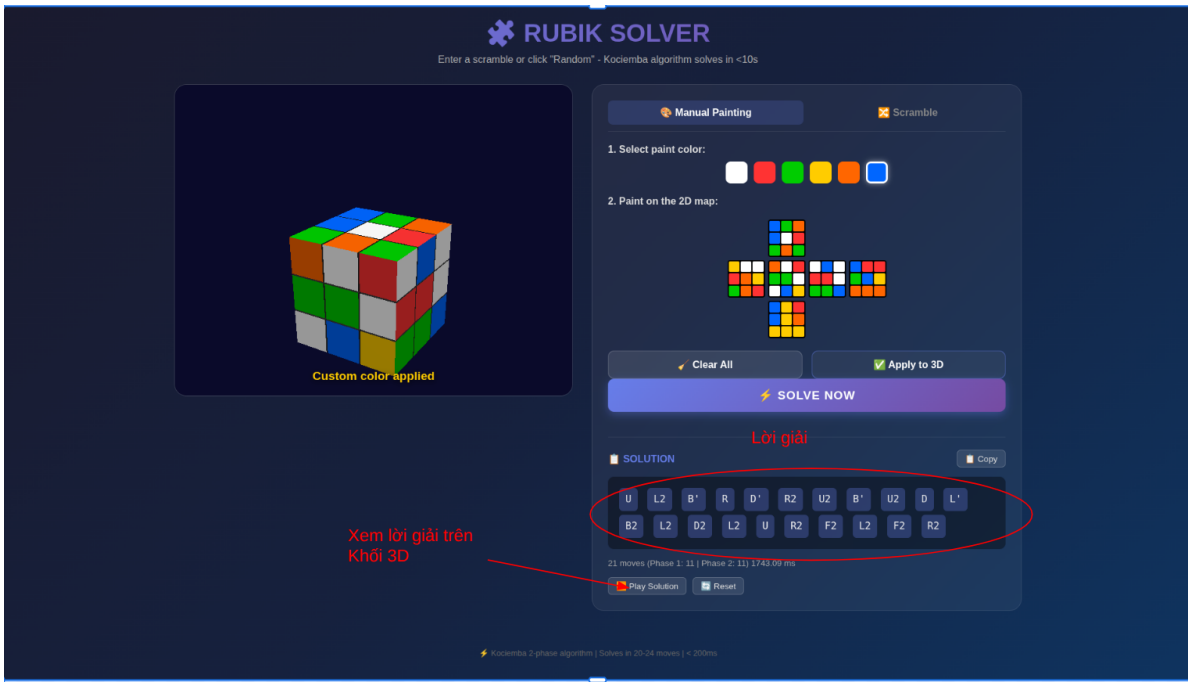


Figure 3.3 Solution display and playback demo

Together, these demo screens show the complete workflow of the system: input a cube state, validate and visualize it, solve it with the Kociemba backend, and replay the solution interactively in the browser.

4 Background Theory

Several algorithmic approaches have been developed over the decades to solve the Rubik's Cube by computer. Below is a brief overview of the most notable ones.

4.1 Thistlethwaite's Algorithm (1981)

Proposed by Morwen Thistlethwaite, this was a landmark algorithm that first demonstrated how group-theoretic decomposition could make the cube tractable. It divides the solving process into four stages, each operating within a progressively smaller subgroup of the cube group. Each stage is solved using a lookup table. Though not optimal, it was groundbreaking as it guaranteed a solution within approximately 52 moves.

4.2 4-List Algorithm (Shamir's Algorithm, 1989)

The 4-list algorithm uses a bidirectional search (meet-in-the-middle approach). It works by storing all 621,649 permutations up to depth 5 in RAM. By multiplying pairs from this list, it generates permutations up to depth 10 from both the solved and scrambled states. Expanding simultaneously, it finds a solution of at most 20 moves where the two searches intersect. While it guarantees a $\leq$20-move solution, it is not designed to find the optimal solution quickly, and its search time is considerably longer than modern algorithms like Kociemba's.

4.3 Korf's Algorithm (1997)

Richard Korf created the first program to solve randomly scrambled cubes optimally. It uses IDA* (Iterative Deepening A*) search with large pattern databases as heuristics. Korf decomposed the cube into three small subproblems: corners only, 6 edges, and the other 6 edges. The number of moves to solve these subproblems serves as a lower bound for the entire cube. Using iterative deepening, branches are pruned when their length combined with the lower-bound heuristic exceeds the current limit. While guaranteed to find optimal solutions, its search time is considerably longer than practical algorithms like Kociemba's.

4.4 Deep Learning Approaches (DeepCubeA)

More recent research explores using reinforcement learning to train neural networks to solve the Rubik's Cube. Models like DeepCubeA learn a value function by self-play and can find solutions using a trained heuristic, somewhat mimicking how a top speedcuber might "intuitively" recognize efficient move sequences. While impressive, these approaches are computationally expensive to train and less predictable than classical algorithms.

4.5 Kociemba's Algorithm (1992) — Used in this project

Proposed by Herbert Kociemba, this algorithm represents the best practical balance between solution quality (near-optimal move count) and computational speed (solves in milliseconds to seconds). It builds upon Thistlethwaite's idea but reduces the decomposition to just two phases, with carefully designed coordinate systems and lookup tables. Nearly all serious speedcubing timers, scramble generators, and solvers in use today are based on this algorithm.

5 Kociemba Algorithm

5.1 Introduction to the Two-Phase Algorithm

The Kociemba algorithm belongs to a family of group-theoretic search methods. The core idea is to exploit the algebraic structure of the cube's symmetry group to decompose the search problem.

Let G denote the group of all possible states of the 3×3 Rubik's Cube (approximately 43 quintillion elements). Kociemba defines a special subgroup:

- G1 = { states where all edges and corners are correctly oriented, AND the 4 middle-layer (UD-slice) edges are within the middle layer }

The algorithm then proceeds in two phases:

- Phase 1: Search from the scrambled state to any state within G1. Using mathematical skills and algorithm, the maximum number of moves for any random state to reach subgroup G1 is 12 moves.
- Phase 2: Search from that G1-state to the fully solved state, using only the restricted move set (reduced to 10 moves) that keeps the cube within G1. The maximum number of moves for phase 2 is calculated to be 18.

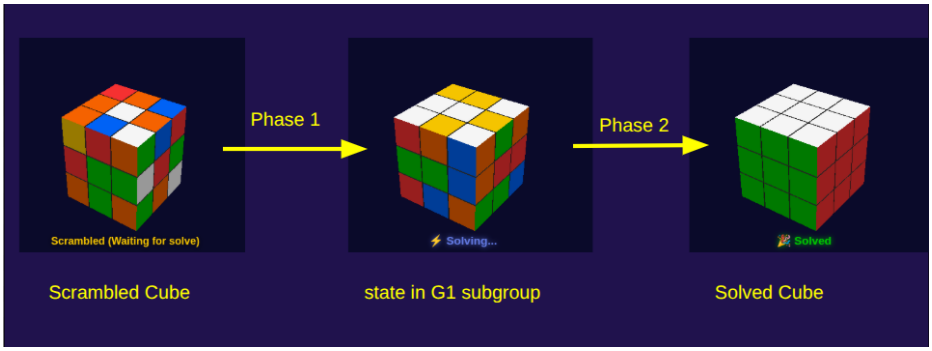


Figure 5.1 2 phase

Why this works so well: By constraining Phase 2 to only moves within G1, the search space shrinks dramatically:

Space	Size
All cube states (G)	∼43 quintillion
States in G1	∼19.5 billion
Phase 1 search space	Manageable with heuristics
Phase 2 search space	Greatly reduced

This decomposition is the key insight that makes Kociemba's algorithm orders of magnitude faster than an exhaustive or purely optimal search.

5.2 Cubie Representation of the Rubik's Cube

To apply the algorithm, the state of the cube must be encoded in a form that is both mathematically precise and computationally efficient. The Cubie Representation serves this purpose.

Rather than tracking the color of each of the 54 individual stickers (facelets), the cubie representation tracks the position and orientation of each of the 20 movable pieces: 8 corner cubies and 12 edge cubies.

5.2.1 Corner Cubies

The 8 corner positions are labeled and assigned integer IDs:

```
URF=0, UFL=1, ULB=2, UBR=3,
DFR=4, DLF=5, DBL=6, DRB=7
```

The state of corners is stored as a list of pairs `(c, o)`:

- `corners[i].c = j` means: corner cubie with ID `j` is currently occupying position `i`.
- `corners[i].o = k` (where k ∈ {0, 1, 2}) means: the corner at position `i` is twisted `k` times clockwise from its canonical orientation.

Canonical orientation for corners: A corner is in orientation 0 if its U-face (white) or D-face (yellow) sticker is facing Up (for top-layer corners) or Down (for bottom-layer corners). Orientations 1 and 2 represent clockwise and counter-clockwise twists, respectively.

5.2.2 Edge Cubies

The 12 edge positions are labeled:

```
UR=0,  UF=1,  UL=2,  UB=3,
DR=4,  DF=5,  DL=6,  DB=7,
FR=8,  FL=9,  BL=10, BR=11
```

The state of edges is stored as pairs `(e, o)`:

- `edges[i].e = j` means: edge cubie `j` is at position `i`.
- `edges[i].o = k` (k ∈ {0, 1}) means: the edge is flipped `k` times from its canonical orientation.

Canonical orientation for edges: Each edge position has a "priority face" (U/D for top/bottom-layer positions; F/B for middle-layer positions). Each edge cubie has a "priority color" (white/yellow for U/D-layer edges; green/blue for middle-layer edges). An edge is in orientation 0 if its priority color faces its position's priority face.

5.2.3 Applying Moves on Cubie Representation

A single move (e.g., R, U', F2) is represented as a permutation + orientation change on the corners and edges. Given a base move matrix for a 90° clockwise turn, composite moves (180° = apply twice; counter-clockwise = apply three times) are computed by composing the transformation matrices.

Pseudocode for composing two corner transformations a and b:

```
function corn_mult(a, b):
    for each position i from 0 to 7:
        prod[i].c = a[ b[i].c ].c
        prod[i].o = (a[ b[i].c ].o + b[i].o) mod 3
    return prod
```

Pseudocode for applying move m to current cube state:

```
function apply_move(cube, move_index):
    base_corner_transform = CORNER_MOVE_MATRIX[face_of(move_index)]
    repeat = repetitions_of(move_index)   # 1, 2, or 3
    final_transform = compose(base_corner_transform, repeat times)
    cube.corners = corn_mult(cube.corners, final_transform)
    cube.edges   = edge_mult(cube.edges,   final_transform)
```

5.2.4 Validity Checks (Parity)

Not every assignment of corners and edges forms a valid, solvable cube. Three parity conditions must hold:

1. Edge Flip Parity: `sum(edge[i].o for all i) mod 2 == 0`
2. Corner Twist Parity: `sum(corner[i].o for all i) mod 3 == 0`
3. Permutation Parity: The permutation parity of the corners must equal the permutation parity of the edges (you cannot swap exactly 2 corners without also swapping 2 edges).

5.3 The Two Phases in Detail

5.3.1 Phase 1: Reaching Group G1

Goal: Transform the scrambled cube into a state where:

1. All 8 corners have orientation 0 (no twist).
2. All 12 edges have orientation 0 (no flip).
3. The 4 UD-slice edges (FR, FL, BL, BR) are located within the middle layer (though not necessarily in their correct positions within it).

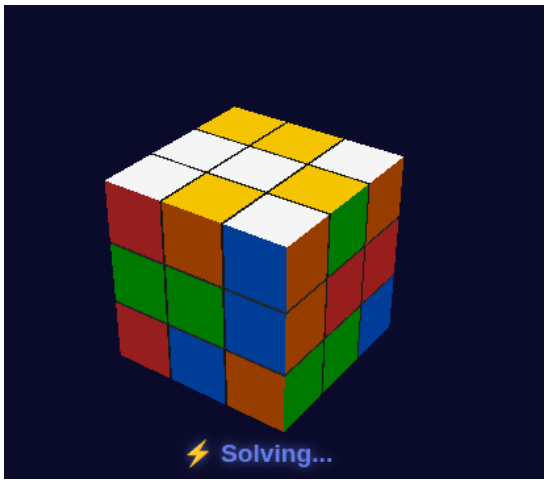


Figure 5.2 Phase 1

Coordinates used in Phase 1:

Coordinate	What it measures	Range
Corner Orientation (twist)	Combined twist state of 8 corners	0 – 2186 (= 3^7 - 1)
Edge Orientation (flip)	Combined flip state of 12 edges	0 – 2047 (= 2^{11} - 1)
UD-Slice (udslice)	Positions of the 4 middle-layer edges	0 – 494 (= $\binom{12}{4}$ - 1)

Why 3^7 and 2^{11}? Due to the parity constraints, the orientation of the last corner (or edge) is always determined by the other seven (or eleven). So only 7 corners and 11 edges are "free," giving $3^7 = 2187$ and $2^{11} = 2048$ states, respectively.

Allowed moves: All 18 standard moves — {U, U2, U', R, R2, R', F, F2, F', D, D2, D', L, L2, L', B, B2, B'}.

Bound on Phase 1 length: Through precomputed pruning tables, the algorithm can determine a lower bound on how many more moves are needed. Empirically, Phase 1 solutions are typically found within 12 moves or fewer.

Search optimization: Although there are 3 coordinates in Phase 1, the pruning table is built from only 2 of them (twist and flip). The UD-slice coordinate is checked at the end of a candidate sequence, not used in the heuristic. This is a deliberate speed-vs-memory tradeoff (discussed further in §3.4).

5.3.2 Phase 2: Solving Within G1

Goal: Starting from the G1-state reached in Phase 1, solve the cube completely by permuting pieces without breaking the G1 conditions.

Coordinates used in Phase 2:

Coordinate	What it measures	Range
Corner Permutation	Arrangement of the 8 corner cubies	0 – 40319 (= 8! - 1)
Edge8 Permutation	Arrangement of 8 U/D-layer edges	0 – 40319 (= 8! - 1)
Edge4 Permutation	Arrangement of 4 middle-layer edges within the slice	0 – 23 (= 4! - 1)

Allowed moves: Only moves that preserve G1 membership — {U, U2, U', D, D2, D', R2, L2, F2, B2}. Notably, R, R', L, L', F, F', B, B' are forbidden because they would flip edges or displace middle-layer edges.

Bound on Phase 2 length: Due to the restricted move set and tighter coordinate spaces, Phase 2 solutions are typically found within 18 moves or fewer, and in practice usually under 12.

5.4 Lookup Tables

Recomputing the full cube state after every move during search will be expensive. Instead, the algorithm relies on two types of precomputed tables.

5.4.1 Move Tables

A move table is a 2D array where:

```
move_table[coord][move_index] = new_coord
```

Given the current coordinate value and a move index (0–17), the table instantly returns the new coordinate value — no cube arithmetic needed during search.

Move tables used in this project:

Table	Dimensions	Description
`twist_move_table`	2187×18	Corner orientation after each move
`flip_move_table`	2048×18	Edge orientation after each move
`udslice_move_table`	495×18	UD-slice position after each move
`corner_perm_move_table`	40320×18	Corner permutation after each move
`edge8_perm_move_table`	40320×18	Edge8 permutation after each move
`edge4_move_table`	24×18	Edge4 permutation after each move

These tables are computed once (a process taking a few minutes on first run) and serialized to disk . On subsequent runs, they are loaded instantly, and lookup for move in these tables takes only O(1)

5.4.2 Pruning Tables

A pruning table (also known as a pattern database) stores, for a given pair of coordinates, the minimum number of moves needed to bring both coordinates to their goal values (0). This value is used as the heuristic `h` in IDA* search.

Construction algorithm (Backward BFS):

```
1. Initialize table with -1 (unvisited) for all entries.
2. Set table[goal_index] = 0.
3. For depth d = 0, 1, 2, ...:
   For each index with table[index] == d:
       For each of the 18 moves:
           Compute the predecessor index (reverse the move).
           If predecessor is unvisited (== -1):
               Set table[predecessor] = d + 1.
4. Repeat until all entries are filled.
```

This BFS from the goal state outward fills in the exact minimum distance for every reachable state, making it an admissible heuristic (never overestimates).

Pruning tables in this project:

Phase 1 pruning table (`phase1_pruning_table.pkl`):

- Combines `twist` and `flip` coordinates.
- Index: `index = twist x 2048 + flip`
- Table size: $2187 \times 2048 \approx 4.5$ million entries.
- `table[index]` = minimum moves to fix both corner orientation and edge orientation.

Phase 2 pruning table (`phase2_pruning_table.pkl`):

- Combines `corner_perm` and `edge8_perm` coordinates.
- Index: `index = c_perm x 20160 + (e8_perm // 2)`
- The division by 2 exploits permutation parity: since the parity of `c_perm` and `e8_perm` are always identical (a fundamental property of the Rubik's Cube group), only half the `e8_perm` values are ever paired with any given `c_perm`. This halves the table size without losing information.
- Table size: $40320 \times 20160 \approx 406$ million entries (compressed from 812 million).
- `table[index]` = minimum moves (within Phase 2's restricted move set) to permute all corners and U/D-layer edges correctly.

Heuristic admissibility and the speed-memory tradeoff:

The Phase 1 heuristic is admissible because fixing corner and edge orientation is a necessary (but not sufficient) condition for solving the cube. Therefore, the table's estimate never exceeds the true move count — IDA* is guaranteed to find the optimal Phase 1 solution.

Ideally, the pruning table would also incorporate the `udslice` coordinate, giving a stronger heuristic. However, this would require approximately 2.2 billion entries ($>$ 2 GB of RAM), which is impractical for a web server environment. The current design deliberately omits `udslice` from the pruning table, sacrificing some pruning power in exchange for feasible memory usage.

The original Kociemba algorithm employs a mathematical trick using the 48 symmetries of the cube (rotations and reflections) to compress the state space. By grouping symmetric states into equivalence classes, the pruning table size can be reduced by a factor of $\sim$16. This optimization theoretically allows building the full 3D pruning table (twist + flip + udslice) while fitting in memory. However, due to its mathematical complexity and the overhead of symmetry reductions during the search phase, this project opts for a simpler, uncompressed 2D table approach.

5.5 IDA* Search Algorithm

IDA* (Iterative Deepening A*) is a memory-efficient variant of A* search. Instead of maintaining an open list of all frontier states in memory, it performs repeated depth-first searches with increasing depth bounds.

Core IDA procedure:

```
function IDA_star(start_state, heuristic):
    bound = heuristic(start_state)
    path  = [start_state]
    loop:
        result = dfs(path, g=0, bound, heuristic)
        if result == FOUND:   return path
        if result == INFINITY: return NOT_FOUND
        bound = result        # raise the bound and retry

function dfs(path, g, bound, heuristic):
    current = path.last
    f = g + heuristic(current)
    if f > bound: return f        # prune: too costly
    if is_goal(current): return FOUND
    min_exceeded = INFINITY
    for each valid move m:
        next_state = apply_move(current, m)
        path.append(next_state)
        result = dfs(path, g + 1, bound, heuristic)
        if result == FOUND: return FOUND
        min_exceeded = min(min_exceeded, result)
        path.pop()
    return min_exceeded
```

IDA applied to Phase 1:

At each search node, the state is described by three coordinates: corner twist (`twist`), edge flip (`flip`), and middle-layer edge position (`udslice`).

The heuristic value `h` (the minimum moves required to fix orientation) is looked up from the 2D Phase 1 pruning table using only `(twist, flip)`.

```
Phase 1 goal check:  twist == 0  AND  flip == 0  AND  udslice == 0
Phase 1 heuristic:   h = pruning_table_p1[ twist * 2048 + flip ]
```

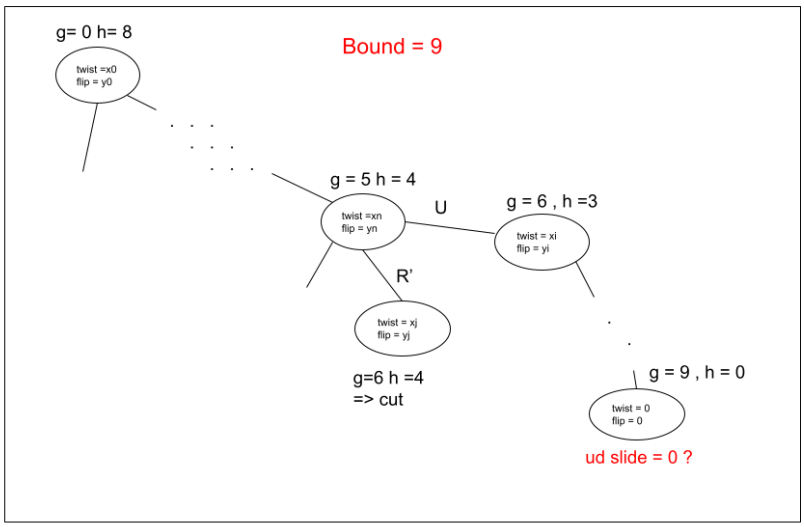


Figure 5.3 IDA

Because the pruning table lacks information about the `udslice` coordinate, `h` is often an underestimate of the true distance to the Phase 1 goal (G1). Consequently, IDA* will frequently explore a path up to the initial bound `h`, find that `twist` and `flip` are 0 but `udslice` is not, and fail the goal check. When this happens, IDA* must iteratively increase the bound (Iterative Deepening) and search deeper until it finds a path that solves all three coordinates simultaneously. (If a full 3D table were used, h would be perfect, and no bound increases would be necessary).

IDA applied to Phase 2:

Phase 2 uses the exact same IDA* skeleton, but operates in a completely different coordinate space and uses a restricted 10-move set (no quarter turns of L, R, F, B). The state is defined by three permutation coordinates: corner permutation (`c_perm`), U/D-layer edge permutation (`e8_perm`), and middle-layer edge permutation (`e4_perm`).

The heuristic is fetched from the Phase 2 pruning table using `c_perm` and `e8_perm`.

```
Phase 2 goal check:  c_perm == 0  AND  e8_perm == 0  AND  e4_perm == 0
Phase 2 heuristic:   h = pruning_table_p2[ c_perm * 20160 + (e8_perm // 2) ]
```

Similar to Phase 1, because the `e4_perm` (the order of the 4 middle-layer edges) is omitted from the heuristic table to save memory, the bound is just a lower estimate. The algorithm relies on the IDA* bound-increasing mechanism to push the search deeper whenever the true distance to the fully solved state exceeds the heuristic estimate.

5.6 Suboptimal Search Strategy

Finding the first valid pair of Phase 1 + Phase 2 solutions does not necessarily yield the shortest overall solution. In practice, the initial combination found by the algorithm is typically around 23 to 25 moves total.Kociemba's algorithm employs an iterative suboptimal search strategy to bring this move count down to a near-optimal length (usually 20 to 22 moves)

Core idea: Once a solution of total length `best` is found, continue searching for Phase 1 solutions of length `p1 + 1`, `p1 + 2`, etc. A slightly longer Phase 1 sequence may lead the cube to a different G1 state from which Phase 2 can be solved in significantly fewer moves.

Example:

- First found: 11 (P1) + 13 (P2) = 24 moves
- Second attempt: 12 (P1) + 10 (P2) = 22 moves
- Third attempt: 13 (P1) + 8 (P2) = 21 moves even better

Intelligent pruning during suboptimal search:

```
If current best total = best_len,
and we are trying a Phase 1 of length p1_new,
then Phase 2 must complete in ≤ (best_len - p1_new - 1) moves.
→ If Phase 2 cannot meet this bound, discard this branch immediately.
```

This constraint prevents wasting time on Phase 1 solutions that cannot possibly improve the current best.

The Latency Tradeoff & The 10-Second Timeout: While suboptimal search significantly improves the solution quality, it comes at a massive computational cost. The runtime can spike from a mere 0.2 seconds (to find the first 24-move solution) to several seconds, or even minutes while hunting for the 20-move solution. In a web server environment, hanging for minutes is unacceptable. Therefore, a 10-second hard timeout mechanism is explicitly implemented in this project's backend. If the suboptimal search cannot finish within 10 seconds, the worker process is forcefully terminated to free up server resources, and a timeout response is returned.

Solution simplification: After each candidate (P1 + P2) pair is assembled, the concatenated move sequence is passed through a simplification function that cancels redundant moves at the junction — for example, `... R U` followed by `U' R' ...` becomes `... (nothing)`.

This suboptimal search strategy is closely related to the Branch-and-Bound (B&B) technique from combinatorial optimization. In B&B, a continuously updated upper bound (the current best solution length) is used to prune branches of the search tree that cannot improve upon it. The iterative structure — trying increasingly long Phase 1 solutions while enforcing

tight upper bounds on Phase 2 — is essentially a best-first branch-and-bound applied to a two-stage decomposition problem.

6 Web Architecture

This section describes the technical architecture of the web application that wraps the Kociemba solver into an interactive, browser-based interface.

6.1 Backend Architecture

The backend is built with FastAPI (Python), following a Layered Architecture that separates concerns into distinct modules.

Technology stack:

- FastAPI — High-performance async web framework
- Pydantic — Data validation and serialization for request/response schemas
- asyncio — Non-blocking concurrency via coroutines and event loop
- multiprocessing — True parallelism and process isolation for the CPU-heavy solver
- uvicorn — ASGI server to run the FastAPI application

6.1.1 Directory Structure

```
rubikweb/
|-- api/
|   |-- __init__.py
|   `-- routes.py           # HTTP endpoints (@router.post("/api/solve"))
|-- core/
|   |-- __init__.py
|   `-- config.py           # Configuration (TIMEOUT, MAX_CONCURRENT_SOLVES)
|-- schemas/
|   |-- __init__.py
|   `-- models.py           # Pydantic schemas (SolveRequest, SolveResponse)
|-- services/
|   |-- __init__.py
|   `-- solver_service.py   # Business logic, timeout management, IPC
|-- kociemba/               # Kociemba algorithm core
|-- static/                 # Frontend assets (HTML, CSS, JS)
`-- main.py                 # FastAPI app entry point
```

6.1.2 Key Architectural Features

Concurrency Control (Semaphore)

The application uses `asyncio.Semaphore` with a limit of 2 concurrent solves. When multiple users submit requests simultaneously, excess requests are queued rather than rejected. This protects server resources on low-spec (free-tier VPS) deployments.

Non-Blocking Event Loop (`asyncio.to_thread`)

The Rubik solver is computationally intensive (it may take 1–5 seconds). Running it directly in the FastAPI event loop would freeze the server, preventing it from responding to any other request. The solution is to offload the computation to a background thread using `asyncio.to_thread()`, keeping the main event loop free.

Process Isolation with Hard Timeout (Multiprocessing)

Even within the background thread, the solver runs inside a child process spawned via Python's `multiprocessing` module (using `fork` context). Results are communicated back through a `Pipe`. This design provides:

- Isolation: A crash or memory error in the solver does not bring down the web server.
- Hard Timeout: The parent monitors the pipe using `poll()`. If the solver does not complete within 10 seconds, the child process is forcefully killed, and an HTTP 408 (Timeout) is returned to the client.

Shared Lookup Tables via Fork

Because the process is forked (not spawned), the child worker inherits the parent's memory space, including all preloaded Kociemba lookup tables. The tables are loaded once at server startup and reused across all solve requests without redundant I/O — a critical performance optimization.

6.1.3 Backend Request Flow

```
[Client] POST /api/solve  {"scramble": "R U R' U' ..."}
    |
    ▼
[Pydantic] Validate SolveRequest schema
    |
    ▼
[asyncio.Semaphore] Acquire slot  (queue if limit reached)
    |
    ▼
[asyncio.to_thread] Offload to background thread
    |
    ▼
[_solve_with_hard_timeout] Spawn child process (fork)
    |
    ├──► [Child] FaceletCube.scramble(scramble_str)
    |           → convert to RubikCube (cubie repr.)
    |           → solve_cube()  [IDA* Phase 1 + Phase 2]
    |           → send result through Pipe
    |
    ▼
[Parent] poll Pipe every 100ms  (hard timeout = 10s)
    ├── Timeout exceeded → process.kill() → HTTP 408
    └── Result received  → unpack SolveResponse
    |
    ▼
[Router] Return JSON  →  HTTP 200
    |
    ▼
[Client] { solution, moves_count, phase1_moves,
           phase2_moves, solving_time_ms, success }
```

Figure 6.1 Backend Request Flow

6.2 Frontend Architecture

The frontend is a vanilla JavaScript single-page application using ES6 Modules for clean code separation. It renders an interactive 3D Rubik's Cube via Three.js.

Technology stack:

- HTML5 + Vanilla CSS — Structure and styling
- ES6 JavaScript Modules — Modular code organization
- Three.js — WebGL-based 3D rendering engine

6.2.1 Frontend File Structure

```
static/
|-- index.html          # Main page (single HTML entry point, two-tab UI)
|-- style.css           # All styling: layout, colors, animations, hover effects
```

```
‘-- main.js            # Entry point --- imports all modules, registers events
      |
      |-- globals.js   # Shared app state, color palette, UI helper functions
      |-- validators.js# Input validation (scramble syntax, facelet validity)
      |-- cube3d.js    # Three.js engine: scene, camera, lights, animation
      ‘-- api.js       # Backend communication (fetch POST to FastAPI)
```

6.2.2 Module Responsibilities

Module	Role
`globals.js`	Single source of truth for app state (`state`), color palette (`COLORS`), API base URL. Shared helpers: `updateStatus()`, `clearSolution()`.
`validators.js`	Pure functions for input validation. `isValidScramble()` checks move syntax. `isValidPieces()` checks that a user-painted facelet layout forms a physically valid, solvable cube (sticker counts, edge flip parity, corner twist parity, permutation parity).
`cube3d.js`	All Three.js logic: scene graph, camera, lighting, OrbitControls (mouse drag to rotate). `rotateLayer()` uses rotation matrices to animate individual layer turns smoothly. Exports `applyMoves()` and `resetCube()`.
`api.js`	Encapsulates all backend calls. `solveRubikApi()` packages input data and posts to the appropriate endpoint. Handles errors and propagates messages to the UI.
`main.js`	The "conductor" — imports all modules, registers all DOM event listeners, orchestrates the full user interaction sequence.

6.2.3 Frontend Exception Handling

Two layers of validation protect the user experience

Client-side validation (before any API call):

- Scramble syntax errors caught by `isValidScramble()` — shown immediately.
- Invalid facelet configurations caught by `isValidPieces()` — checks parity rules directly in the browser without a server round-trip.

API error handling (in `api.js`):

- HTTP 408 (timeout) → notifies user that the solver timed out.
- HTTP 400 (invalid cube state detected server-side) → displays the backend's mathematical error.
- Network failure → generic connectivity error message.

6.2.4 Frontend Architecture Diagram

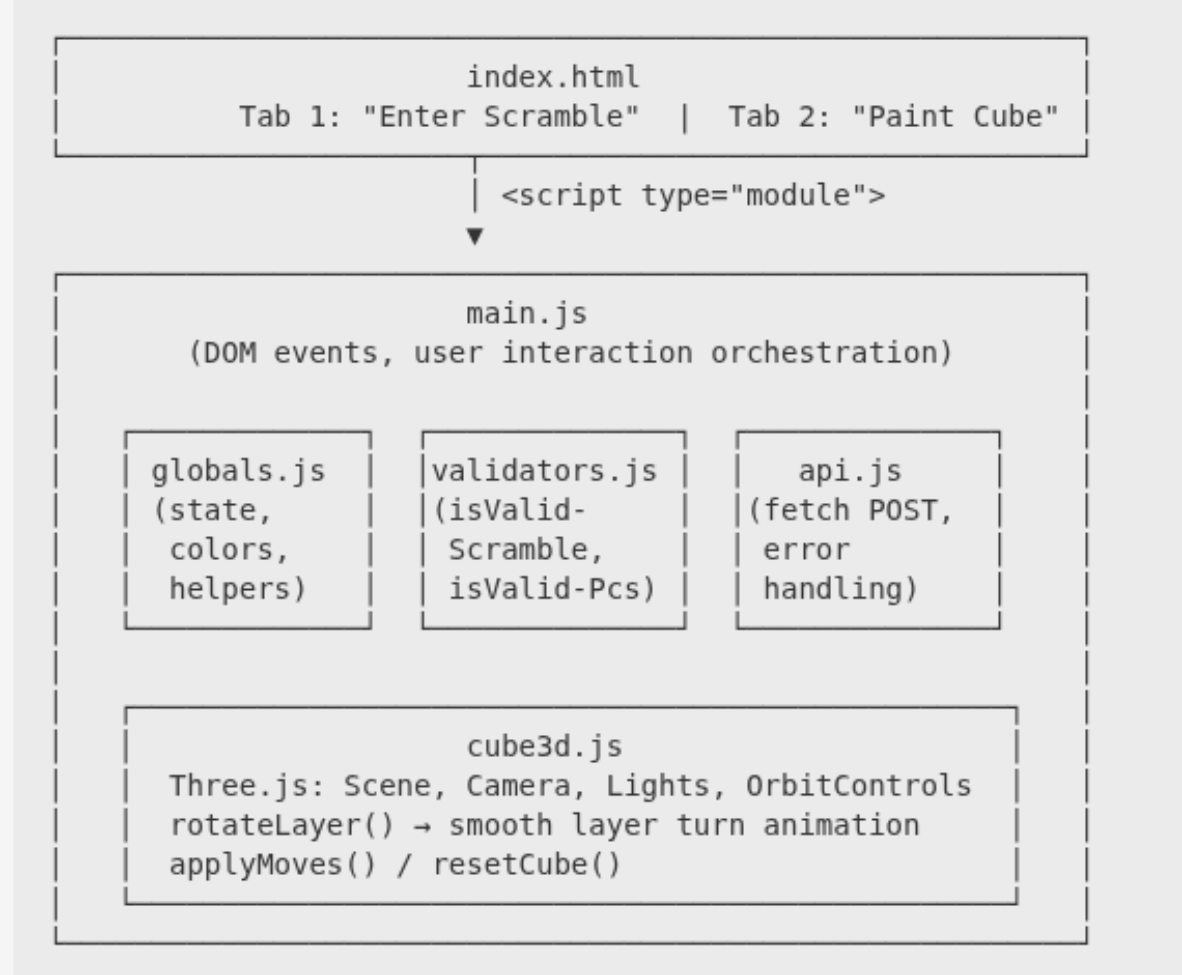


Figure 6.2 Frontend Architecture Diagram

6.2.5 Frontend User Flow (Color Painting Mode)

```
User selects a color → clicks a cell in the 2D Facelet Grid
    |
    ▼
main.js updates state.faceletColors  (via globals.js)
    |
    ▼  [Click "Apply to 3D"]
validators.js: isValidPieces(faceletColors)
    ├── Invalid → show error alert  (no API call)
    └── Valid   →
          cube3d.js: resetCube() → recolor 3D cube materials
          |
          ▼  [Click "Solve Now"]
          api.js: POST /api/solve-by-facelet  { facelet_string }
          |
          ▼
          Receive { solution, moves_count, ... }
          Update DOM: display move list and statistics
          |
          ▼  [Click "View Solution"]
          main.js iterates solution moves:
              cube3d.js.rotateLayer(move) → animated 3D playback
```

Figure 6.3 Frontend User Flow (Color Painting Mode)

6.3 Overall Application Flow

complete journey from user action to animated 3D solution:

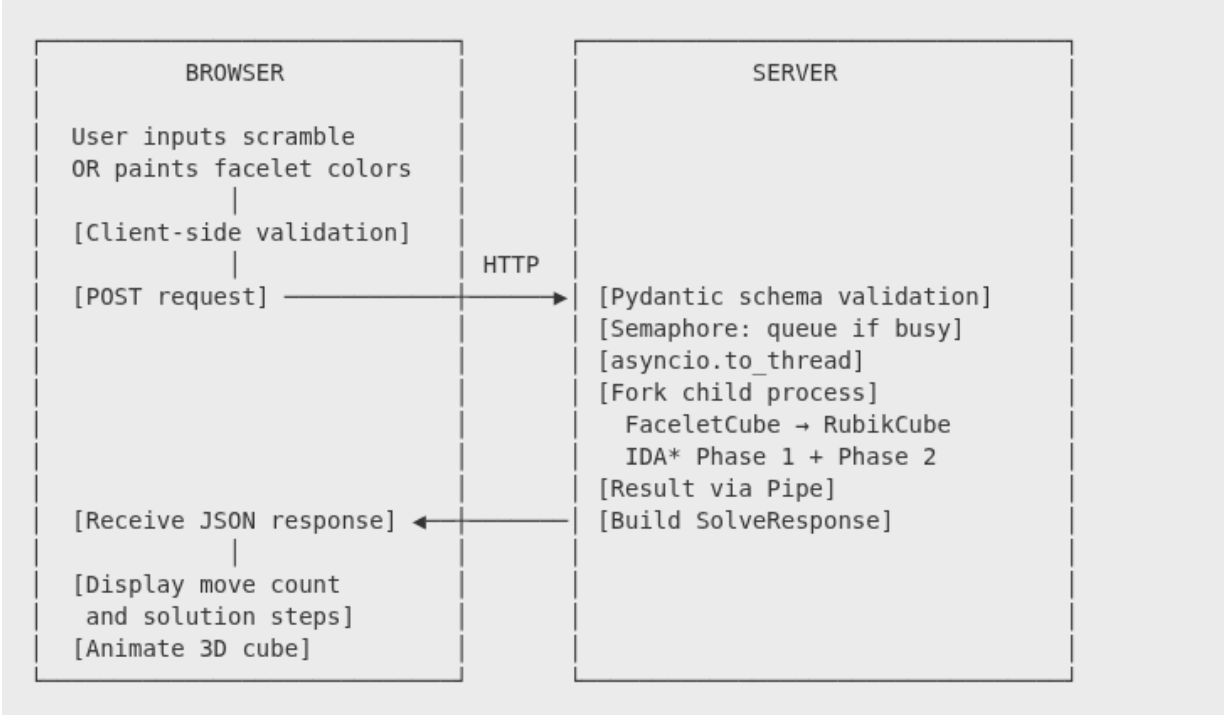


Figure 6.4 Overall Application Flow

The application offers two input modes:

- Scramble Mode: The user types a standard WCA-format move sequence (e.g., `R U R' F2 L D2 ...`). The backend applies this scramble to a solved cube in cubie representation, then runs the solver.
- Facelet Painting Mode: The user paints each sticker of the cube on a 2D net diagram. The 54-character facelet string is sent to the backend, converted to cubie representation, validated for parity, and then solved.

In both cases, the result is an animated step-by-step playback of the solution rendered on the interactive 3D WebGL cube in the browser.

7 Future Development

While the current implementation provides a robust, fully functioning solver, there are several avenues for future enhancements:

1. Symmetry Reductions: Implementing the 48 symmetries of the cube to compress the pruning tables. This would allow the inclusion of the `udslice` coordinate in Phase 1 (forming a full 3D pruning table) without exceeding memory limits, significantly strengthening the heuristic and reducing the number of nodes explored in the search tree.

2. WebAssembly (WASM) Solver: Currently, the solving algorithm runs entirely on the backend server. Porting the Kociemba core to WebAssembly (e.g., using C++ or Rust) would allow the solver to run directly in the client's browser, eliminating network latency and eliminating server CPU load entirely.

3. Advanced 3D Controls: Enhancing the 3D visualization by allowing users to manually rotate individual layers with mouse swipes (drag-to-rotate), rather than just viewing the playback or painting colors.

4. Camera Input (Computer Vision): Adding a feature to scan a physical Rubik's Cube using a webcam or mobile camera. Computer vision algorithms (like OpenCV) could detect the colors automatically, filling out the facelet layout without manual user input.

End of Report